MODULE *framing*

Tor transport stream-framing specification.

The *Tor* transport carries the stream layer over a SINGLE ordered bytestream per connection, using a QUIC-modelled framing layer ("Stream Framing"): STREAM frames append data to streams (FIN flag finishes a direction), opening is implicit in the first frame of a new stream *ID*, and flow control is cumulative — *MAX_DATA* bounds total bytes across all streams, and *MAX_STREAMS_UNI* bounds the total count of opened unidirectional streams, raised by absolute-valued frames. The initial credits come from the connection preamble. This is the least implementation-tested text of the draft: Zebra's framing layer was removed as unreachable code.

The model: a sender S delivers one bulk record of *Total* bytes on one stream, and separately keeps one announcement stream open, finishing and replacing it (as "Announcement Streams" allows and this repository's Finding 2 recommends), *MaxReplace* times. Frames travel in order; credit grants travel back in order. Three policy switches:

GrantPerRecord	the receiver raises *MAX_DATA* only after processing a COMPLETE record (record-granularity processing), rather than as bytes arrive.
RaiseOnClose	the receiver raises *MAX_STREAMS_UNI* as streams finish, *QUIC*'s way of maintaining a concurrency limit expressed cumulatively.
SenderReadsConcurrent	the sender reads the preamble's stream-limit field as a CONCURRENT limit (the preamble's own word) instead of the cumulative count the framing section defines.

Findings this module reproduces (see . . /documents/v2-*modeling.md*):

- The draft mandates a minimum *initial_max_stream_data* that covers a maximum record, but sets NO minimum for the connection-level *initial_max_data*. A peer advertising less than one record of connection credit, talking to a record-granularity receiver — both conforming — wedges the connection forever (*framing_wedge.cfg*); the same credits with byte-granularity granting complete fine (*framing_perframe.cfg*).
  - The preamble calls the stream-limit fields "concurrent"; the framing section defines them as cumulative. The two readings are both implementable and disagree: a cumulative-reading sender stalls silently when the receiver never raises the limit (*framing_noraise.cfg*), and a concurrent-reading sender is disconnected with *PROTOCOL_ERROR* by a cumulative-enforcing receiver (*framing_concurrent.cfg*) — honest peers reading different sentences of the same section.
  - *StrictSingletonSafeHere*: on this transport the announcement-stream replacement race of Finding 1 CANNOT occur — the pipe is ordered, so the old stream's FIN always precedes the replacement's first frame. The literal singleton rule is safe on *Tor* and unsafe on *QUIC*, which may explain how it was written.

EXTENDS *Naturals*, *Sequences*, *FiniteSets*

CONSTANT *Total*	bytes of the bulk record (*max* record abstracted)
CONSTANT *InitMaxData*	preamble *initial_max_data* (connection credit)
CONSTANT *InitMaxStreamsUni*	preamble *initial_max_streams_uni*
CONSTANT *MaxReplace*	announcement stream replacements S performs
CONSTANT *GrantPerRecord*	see module comment
CONSTANT *RaiseOnClose*	see module comment
CONSTANT *SenderReadsConcurrent*	see module comment

$BulkSid \triangleq 0$

$AnnSid(k) \triangleq k$ announcement streams are $1 \,..\, 1 + MaxReplace$, opened in order

VARIABLES

$pipe$,	ordered frames $S \to R$
$back$,	ordered credit frames $R \to S$
s_sent,	bulk bytes S has sent
s_data_lim,	connection-level credit S may use (last MAX_DATA, cumulative bytes)
s_str_lim,	cumulative count of uni streams S may open (last $MAX\_STREAMS\_UNI$)
s_opened,	uni streams S has opened so far
s_finned,	uni streams S has finished so far
$r\_delivered$,	bulk bytes R has processed
$r\_granted$,	connection credit R has advertised
r_str_lim,	stream-count limit R has advertised
r_opened,	uni streams R has observed opening
r_ann_open,	announcement sids R sees open (opened, no FIN yet)
$closed$	"none" | "PROTOCOL_ERROR"

$vars \triangleq \langle pipe,\ back,\ s\_sent,\ s\_data\_lim,\ s\_str\_lim,\ s\_opened,\ s\_finned,$
$r\_delivered,\ r\_granted,\ r\_str\_lim,\ r\_opened,\ r\_ann\_open,\ closed\rangle$

$NoCode \triangleq$ "none"

Frames: STREAM open of an announcement stream (length 0, abstracted),
its FIN, one bulk data byte, and the two credit-raising frames.

$OpenF(k) \triangleq [t \mapsto$ "open", $sid \mapsto k]$
$FinF(k) \triangleq [t \mapsto$ "fin", $sid \mapsto k]$
$ByteF \triangleq [t \mapsto$ "byte", $sid \mapsto BulkSid]$
$MaxDataF(m) \triangleq [t \mapsto$ "max_data", $max \mapsto m]$
$MaxStreamsF(m) \triangleq [t \mapsto$ "max_streams", $max \mapsto m]$

$Init \triangleq$
$\land\ pipe = \langle\rangle \land back = \langle\rangle$
$\land\ s\_sent = 0 \land s\_data\_lim = InitMaxData$
$\land\ s\_str\_lim = InitMaxStreamsUni \land s\_opened = 0 \land s\_finned = 0$
$\land\ r\_delivered = 0 \land r\_granted = InitMaxData$
$\land\ r\_str\_lim = InitMaxStreamsUni \land r\_opened = 0 \land r\_ann\_open = \{\}$
$\land\ closed = NoCode$

S sends the next byte of the bulk record, within the connection credit
its peer has granted (the per-stream credit never binds: the draft
mandates *initial_max_stream_data* of at least a full record).

$SendBulkByte \triangleq$
$\land\ closed = NoCode$
$\land\ s\_sent < Total$
$\land\ s\_sent < s\_data\_lim$
$\land\ s\_sent' = s\_sent + 1$
$\land\ pipe' = Append(pipe,\ ByteF)$

$\land$ UNCHANGED $\langle back,\ s\_data\_lim,\ s\_str\_lim,\ s\_opened,\ s\_finned,$
$r\_delivered,\ r\_granted,\ r\_str\_lim,\ r\_opened,\ r\_ann\_open,\ closed\rangle$

S opens its announcement stream, or a replacement after finishing the previous one. What "within the stream limit" means is the switch: the cumulative count the framing section defines, or the concurrent count the preamble's field description suggests.

$OpenAnn \triangleq$
$\land\ closed = NoCode$
$\land\ s\_opened = s\_finned$ previous announcement stream finished
$\land\ s\_opened \leq MaxReplace$ the initial stream plus *MaxReplace* replacements
$\land$ IF $SenderReadsConcurrent$
THEN $(s\_opened - s\_finned) < s\_str\_lim$
ELSE $s\_opened < s\_str\_lim$
$\land\ s\_opened' = s\_opened + 1$
$\land\ pipe' = Append(pipe,\ OpenF(AnnSid(s\_opened + 1)))$
$\land$ UNCHANGED $\langle back,\ s\_sent,\ s\_data\_lim,\ s\_str\_lim,\ s\_finned,$
$r\_delivered,\ r\_granted,\ r\_str\_lim,\ r\_opened,\ r\_ann\_open,\ closed\rangle$

S finishes its open announcement stream, intending to replace it.

$FinAnn \triangleq$
$\land\ \ closed = NoCode$
$\land\ \ s\_opened = s\_finned + 1$
$\land\ \ s\_finned' = s\_finned + 1$
$\land\ \ pipe' = Append(pipe,\ FinF(AnnSid(s\_opened)))$
$\land\ \ $UNCHANGED $\langle back,\ s\_sent,\ s\_data\_lim,\ s\_str\_lim,\ s\_opened,$
$r\_delivered,\ r\_granted,\ r\_str\_lim,\ r\_opened,\ r\_ann\_open,\ closed\rangle$

R processes the next frame, in order. A bulk byte may trigger a *MAX_DATA* raise (immediately, or only once the record is complete, per the switch); an open is checked against the cumulative stream limit and the strict singleton reading; a FIN may trigger a *MAX_STREAMS* raise.

$RecvFrame \triangleq$
$\land\ closed = NoCode$
$\land\ pipe \neq \langle\rangle$
$\land$ LET $f \triangleq Head(pipe)$ IN
$\lor\ \land\ f.t =$ "byte"
$\land\ r\_delivered' = r\_delivered + 1$
$\land$ LET $done \triangleq r\_delivered + 1 \geq Total$
$grant \triangleq$ IF $GrantPerRecord$ THEN $done$ ELSE TRUE
IN $\land\ r\_granted' =$ IF $grant$ THEN $Total$ ELSE $r\_granted$
$\land\ back' =$ IF $grant \land r\_granted < Total$
THEN $Append(back,\ MaxDataF(Total))$ ELSE $back$
$\land\ pipe' = Tail(pipe)$
$\land$ UNCHANGED $\langle s\_sent,\ s\_data\_lim,\ s\_str\_lim,\ s\_opened,\ s\_finned,$
$r\_str\_lim,\ r\_opened,\ r\_ann\_open,\ closed\rangle$

$\lor \land f.t = \text{“open”}$
$\quad \land \lor \land r\_opened + 1 > r\_str\_lim$

the framing section's cumulative limit is exceeded:
a connection error of type *PROTOCOL_ERROR*

$\qquad \land closed' = \text{“PROTOCOL\_ERROR”}$
$\qquad \land \text{UNCHANGED } \langle pipe,\ back,\ s\_sent,\ s\_data\_lim,\ s\_str\_lim,$
$\qquad\qquad s\_opened,\ s\_finned,\ r\_delivered,\ r\_granted,$
$\qquad\qquad r\_str\_lim,\ r\_opened,\ r\_ann\_open\rangle$
$\quad \lor \land r\_opened + 1 \leq r\_str\_lim$
$\qquad \land r\_opened' = r\_opened + 1$
$\qquad \land r\_ann\_open' = r\_ann\_open \cup \{f.sid\}$
$\qquad \land pipe' = Tail(pipe)$
$\qquad \land \text{UNCHANGED } \langle back,\ s\_sent,\ s\_data\_lim,\ s\_str\_lim,\ s\_opened,$
$\qquad\qquad s\_finned,\ r\_delivered,\ r\_granted,\ r\_str\_lim,\ closed\rangle$
$\lor \land f.t = \text{“fin”}$
$\quad \land r\_ann\_open' = r\_ann\_open \setminus \{f.sid\}$
$\quad \land \text{LET } raise \triangleq RaiseOnClose$
$\quad\ \text{IN} \quad \land r\_str\_lim' = \text{IF } raise \text{ THEN } r\_str\_lim + 1 \text{ ELSE } r\_str\_lim$
$\qquad\quad \land back' = \text{IF } raise \text{ THEN } Append(back,\ MaxStreamsF(r\_str\_lim + 1)) \text{ ELSE } back$
$\quad \land pipe' = Tail(pipe)$
$\quad \land \text{UNCHANGED } \langle s\_sent,\ s\_data\_lim,\ s\_str\_lim,\ s\_opened,\ s\_finned,$
$\qquad\qquad r\_delivered,\ r\_granted,\ r\_opened,\ closed\rangle$

S processes the next credit frame; limits never decrease.

$RecvBack \triangleq$
$\quad \land closed = NoCode$
$\quad \land back \neq \langle\rangle$
$\quad \land \text{LET } f \triangleq Head(back) \text{IN}$
$\qquad \land s\_data\_lim' = \text{IF } f.t = \text{“max\_data”} \land f.max > s\_data\_lim \text{ THEN } f.max \text{ ELSE } s\_data\_lim$
$\qquad \land s\_str\_lim' = \text{IF } f.t = \text{“max\_streams”} \land f.max > s\_str\_lim \text{ THEN } f.max \text{ ELSE } s\_str\_lim$
$\quad \land back' = Tail(back)$
$\quad \land \text{UNCHANGED } \langle pipe,\ s\_sent,\ s\_opened,\ s\_finned,$
$\qquad\qquad r\_delivered,\ r\_granted,\ r\_str\_lim,\ r\_opened,\ r\_ann\_open,\ closed\rangle$

$Next \triangleq SendBulkByte \lor OpenAnn \lor FinAnn \lor RecvFrame \lor RecvBack$

$Spec \triangleq Init \land \Box[Next]_{vars} \land \text{WF}_{vars}(Next)$

Liveness: the bulk record is eventually delivered, and every replacement announcement stream eventually opens.

$BulkDelivered \triangleq \Diamond(r\_delivered = Total)$
$ReplacementsComplete \triangleq \Diamond(s\_opened = MaxReplace + 1)$

Safety invariants.

$$
\begin{aligned}
&\mathit{TypeOK} \triangleq \\
&\quad \land \quad \mathit{s\_sent} \in 0 \,..\, \mathit{Total} \land \mathit{r\_delivered} \in 0 \,..\, \mathit{Total} \\
&\quad \land \quad \mathit{s\_opened} \in 0 \,..\, (\mathit{MaxReplace} + 1) \land \mathit{s\_finned} \in 0 \,..\, \mathit{s\_opened} \\
&\quad \land \quad \mathit{closed} \in \{\mathit{NoCode},\ \text{“PROTOCOL\_ERROR”}\}
\end{aligned}
$$

Finding 1's race is impossible on this transport: the pipe is ordered, so a replacement's open frame is never observed before the previous stream's FIN — the receiver never sees two announcement streams open.

$$\mathit{StrictSingletonSafeHere} \triangleq \mathit{Cardinality}(\mathit{r\_ann\_open}) \leq 1$$

No interleaving of these conforming-but-divergent peers should end in a protocol error.

$$\mathit{NoHonestProtocolError} \triangleq \mathit{closed} = \mathit{NoCode}$$